

Institute of Engineering & Management, Kolkata

Department of Computer Science and Engineering

Python API Automation Framework

Requests + Behave BDD + Allure Reporting

Capstone Project Report

Selenium / API Testing Coursework

Submitted by:

Simantini Das

Mentor:

Prof. Sourav Malakar

September 26, 2026

Contents

Contents	2
1. Introduction	3
1.1 Objective	3
1.2 Scope	3
1.3 Tools and Technologies Used	3
2. System Design and Architecture	3
2.1 Layered Architecture	3
2.2 Project Folder Structure	4
3. Implementation Details	4
3.1 Configuration Layer	4
3.2 Base API Client	5
3.3 Endpoint-Specific Clients	5
3.4 JSON Schema Validation	6
3.5 BDD Test Scenarios	6
3.6 Allure Reporting	7
4. Testing and Results	7
4.1 Test Execution	7
4.2 Dry-Run Verification	7
4.3 Test Scenarios Covered	7
4.4 Screenshots	8
4.4.1 Console Output — Behave Test Run	8
4.4.2 Allure Report — Overview Dashboard	8
4.4.3 Allure Report — Suites	9
4.4.4 Allure Report — Behaviors	9
4.4.5 Allure Report — Graphs	10
5. Conclusion	10
5.1 Summary	10
5.2 Future Enhancements	10
5.3 Repository	10

1. Introduction

1.1 Objective

The objective of this project is to design and implement a complete API Test Automation Framework using Python. The framework demonstrates industry-standard practices for testing REST APIs, including:

- Sending and validating HTTP requests using the requests library
- Behaviour-Driven Development (BDD) test design using Behave
- Structured, human-readable test reporting using Allure
- Authentication simulation via a login-verification endpoint
- JSON Schema (contract) validation of API responses
- Reusable, modular framework architecture, proven against two independent APIs
- Resilience handling via an automatic retry mechanism for transient failures

1.2 Scope

The framework targets the User Management API hosted at <https://automationexercise.com/api> as the primary system under test, covering the full account lifecycle: account creation, login verification, profile retrieval, profile update, and account deletion. To demonstrate that the framework's design is genuinely reusable and not hardcoded to a single service, the same client architecture is also pointed at a second, independent API — <https://jsonplaceholder.typicode.com/> — with no code changes, only a configuration switch.

1.3 Tools and Technologies Used

Tool / Library	Purpose
Python 3.x	Core programming language
Requests	HTTP client used to call REST endpoints
Behave	BDD framework for Gherkin-style test scenarios
Allure (allure-behave)	Rich, interactive HTML test reports
jsonschema	Contract / schema validation of API responses
PyYAML	Loading environment configuration from YAML
Custom retry decorator	Automatic retry on connection errors, timeouts, and 5xx responses

2. System Design and Architecture

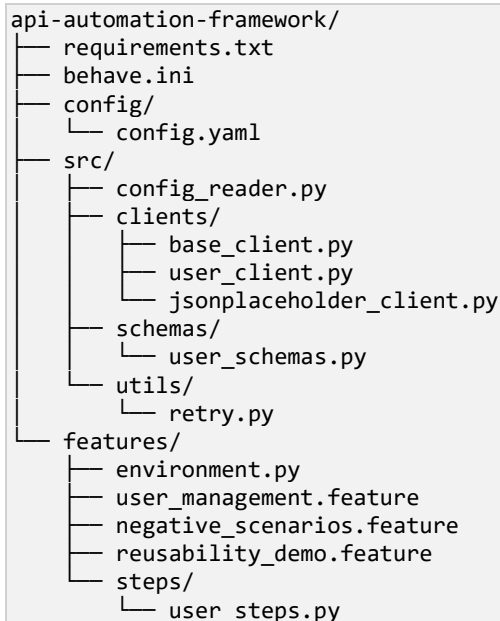
2.1 Layered Architecture

The framework follows a layered design so that responsibilities are cleanly separated and the codebase stays maintainable and reusable:

- Feature Layer — Gherkin .feature files describing test scenarios in plain English.
- Step Definition Layer — Python glue code that maps Gherkin steps to framework calls.
- API Client Layer — Reusable classes wrapping the requests library for each API resource (UserClient, JSONPlaceholderUserClient).
- Base Client / Utility Layer — Shared logic for sessions, timeouts, retries, response timing, and schema validation.



2.2 Project Folder Structure



3. Implementation Details

3.1 Configuration Layer

All environment-specific values (base URL, timeout, expected response-time threshold) are centralised in config/config.yaml. The active environment is resolved by src/config_reader.py, which checks, in order: an explicit value passed via behave -D env=<name>, the API_ENV environment variable, and finally a

default_env value in the YAML file. This allows the same framework to be pointed at a different API purely through configuration.

```
default_env: automationexercise

environments:
  automationexercise:
    base_url: "https://automationexercise.com/api"
    timeout: 15
    max_response_time_ms: 3000

  jsonplaceholder:
    base_url: "https://jsonplaceholder.typicode.com"
    timeout: 15
    max_response_time_ms: 2000
```

3.2 Base API Client

The BaseAPIClient class wraps the requests library and provides shared behaviour to every resource-specific client built on top of it:

- A shared requests.Session for connection reuse and common headers
- Centralised timeout handling
- Automatic retries on connection errors, timeouts, and 5xx responses
- Response-time measurement for every call
- Storage of the last request/response pair for Allure logging

```
@retry_on_failure(max_attempts=3, backoff_seconds=1)
def _send(self, method, endpoint, **kwargs):
    url = self._build_url(endpoint)
    kwargs.setdefault("timeout", self.timeout)

    self.last_request = {
        "method": method, "url": url,
        "params": kwargs.get("params"),
        "data": kwargs.get("data"),
    }

    start = time.perf_counter()
    response = self.session.request(method, url, **kwargs)
    self.last_response_time_ms = (time.perf_counter() - start) * 1000
    self.last_response = response
    return response
```

3.3 Endpoint-Specific Clients

UserClient (targeting automationexercise.com) and JSONPlaceholderUserClient (targeting jsonplaceholder.typicode.com) both inherit from BaseAPIClient and expose resource-specific methods, so step definitions never call the requests library directly:

```
class UserClient(BaseAPIClient):
    def create_account(self, payload):
```

```

        return self.post("createAccount", data=payload)

    def verify_login(self, email=None, password=None):
        payload = {}
        if email is not None: payload["email"] = email
        if password is not None: payload["password"] = password
        return self.post("verifyLogin", data=payload)

    def get_user_by_email(self, email):
        return self.get("getUserDetailByEmail", params={"email": email})

```

3.4 JSON Schema Validation

A key feature of this framework is contract testing — verifying that API responses match an expected structure (required fields, correct data types) rather than only checking status codes. This is handled with the `jsonschema` library, with schema definitions stored in `src/schemas/user_schemas.py`.

```

USER_DETAIL_SCHEMA = {
    "type": "object",
    "required": ["responseCode", "user"],
    "properties": {
        "responseCode": {"type": "integer"},
        "user": {
            "type": "object",
            "required": ["email", "first_name", "last_name"],
        },
    },
}

```

3.5 BDD Test Scenarios

Test cases are written in Gherkin syntax so they remain readable to non-technical stakeholders. The following scenario chains the full account lifecycle end-to-end:

```

Scenario: Successfully manage the full user account lifecycle
    Given I have a new user's registration details
    When I create the user account
    Then the account should be created successfully
    And the response time should be below 3000 ms

    When I verify login with the created user's credentials
    Then the login should be verified successfully

    When I update the user's account information
    Then the account should be updated successfully

    When I delete the user's account
    Then the account should be deleted successfully

```

3.6 Allure Reporting

Every request and response is automatically attached to the Allure report as a text artifact via a Behave `after_step` hook in `features/environment.py`, making it easy to debug failing scenarios without re-running tests. Each attachment includes the HTTP method, URL, request payload, response status code, response time, and response body.

4. Testing and Results

4.1 Test Execution

Tests were executed using the following command, which runs all feature files and simultaneously writes results for Allure via the formatter configured in `behave.ini`:

```
behave
```

The Allure HTML report was then generated and viewed with:

```
allure serve allure-results
```

4.2 Dry-Run Verification

Before full execution, a dry run was performed to confirm that every Gherkin step correctly binds to a step definition:

```
behave --dry-run
```

All steps across the three feature files resolved successfully with zero undefined steps before the framework was run against the live APIs.

4.3 Test Scenarios Covered

Scenario	Type	Result	Duration
Successfully manage the full user account lifecycle	Positive (end-to-end CRUD)	Passed	3s 512ms
Verify login fails when the email field is missing	Negative	Passed	2s 342ms
Verify login fails when the password field is missing	Negative	Passed	348ms
Verify login fails for a user that does not exist	Negative	Passed	365ms
Fetch a user from JSONPlaceholder using the same framework	Reusability demo	Passed	862ms

Final result summary: 3 features passed, 5 scenarios passed, 26 steps passed — 0 failed, 0 skipped (100% pass rate), completed in approximately 4.0 seconds.

4.4 Screenshots

4.4.1 Console Output — Behave Test Run

```
(venv) PS C:\Users\siman\Desktop\selenium_project\selenium_project\api-automation-framework> behave
# features/steps/user_steps.py:138
Then the response code should be 400 # features/steps/user_steps.py:143 # features/steps/user_steps.py:143 0.000s Then the response code should
And the response message should be "Bad request, email or password parameter is missing in POST request." # features/steps/user_steps.py:151 0.000s And the response message sho
d request, email or password parameter is missing in POST request." # features/steps/user_steps.py:151

Scenario: Verify login fails for a user that does not exist # features/negative_scenarios.feature:17
Given a set of credentials that do not belong to any registered user # features/steps/user_steps.py:130
And the response message should be "User not found!" # features/steps/user_steps.py:151 0.001s
And the response message should be "User not found!" # features/steps/user_steps.py:151 0.001s
And the response message should be "User not found!" # features/steps/user_steps.py:151

Feature: Framework reusability across a second API # features/reusability_demo.feature:1
As a QA lead
I want the same client/config/reporting design to work against another API
So that the framework is proven reusable, not hardcoded to one serviceFeature: Framework reusability across a second API # features/reusability_demo.feature:1
As a QA lead
I want the same client/config/reporting design to work against another API
So that the framework is proven reusable, not hardcoded to one service
Scenario: Fetch a user from JSONPlaceholder using the same framework # features/reusability_demo.feature:6
When I fetch user 2 from JSONPlaceholder # features/steps/user_steps.py:173
Then the JSONPlaceholder response should match the user schema # features/steps/user_steps.py:178 0.005s
Then the JSONPlaceholder response should match the user schema # features/steps/user_steps.py:178 0.005s
Then the JSONPlaceholder response should match the user schema # features/steps/user_steps.py:178

Feature: User Account Management # features/user_management.feature:1
As a QA engineer
I want to automate the full lifecycle of a user account
So that I can verify the User Management API behaves correctly end-to-endFeature: User Account Management # features/user_management.feature:1
As a QA engineer
I want to automate the full lifecycle of a user account
So that I can verify the User Management API behaves correctly end-to-end
Scenario: Successfully manage the full user account lifecycle # features/user_management.feature:6
Given I have a new user's registration details # features/steps/user_steps.py:20
Then the account should be deleted successfully # features/steps/user_steps.py:111 0.004s
Then the account should be deleted successfully # features/steps/user_steps.py:111 0.004s
Then the account should be deleted successfully # features/steps/user_steps.py:111
3 features passed, 0 failed, 0 skipped
5 scenarios passed, 0 failed, 0 skipped
26 steps passed, 0 failed, 0 skipped
Took 0min 4.024s
```

Figure 4.1: Behave console output showing the final scenario/step summary

4.4.2 Allure Report — Overview Dashboard

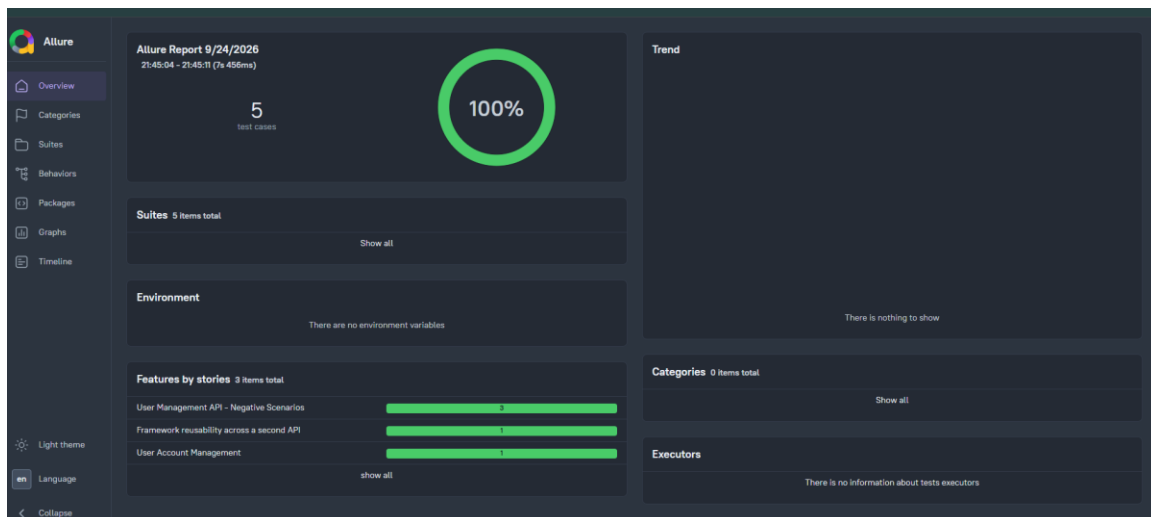


Figure 4.2: Allure report overview dashboard — 5 test cases, 100% pass rate

4.4.3 Allure Report — Suites

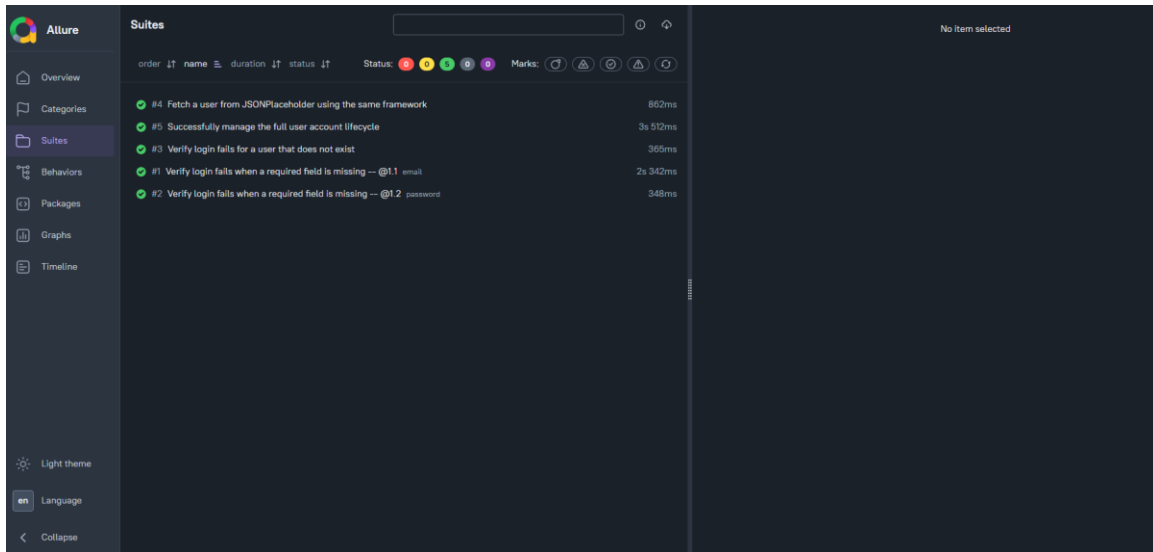


Figure 4.3: Allure report — individual scenario results and durations

4.4.4 Allure Report — Behaviors

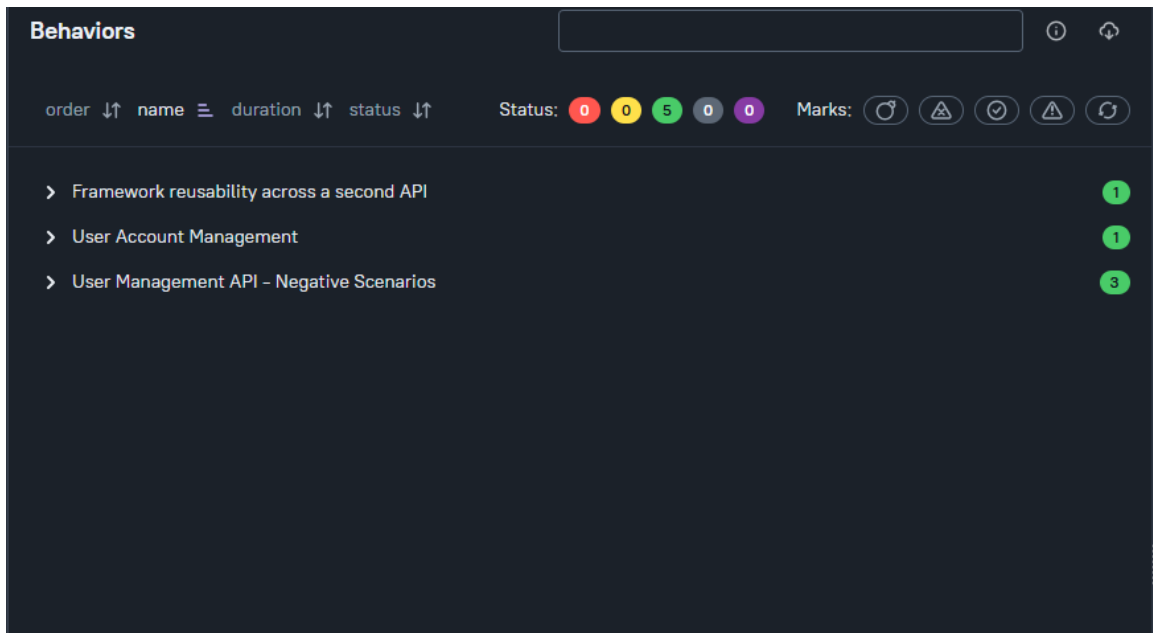


Figure 4.4: Allure report — scenarios grouped by feature (behaviour view)

4.4.5 Allure Report — Graphs

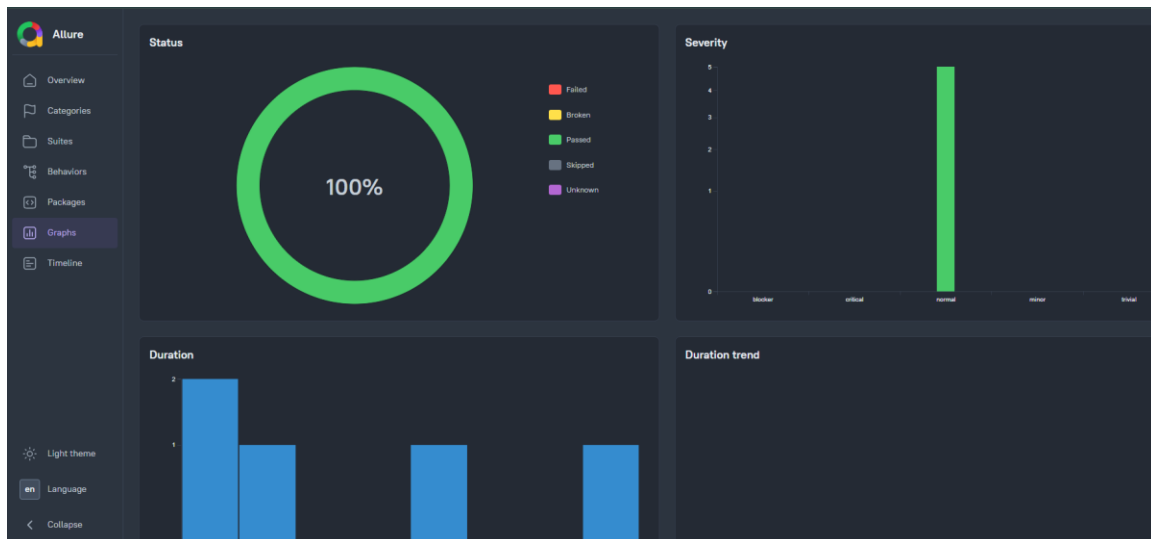


Figure 4.5: Allure report — status, severity, and duration graphs

5. Conclusion

5.1 Summary

This project successfully implements a reusable, layered API test automation framework combining Requests, Behave, and Allure. It demonstrates full account-lifecycle CRUD testing, login-verification (authentication-style) handling, and JSON schema-based contract validation — going beyond simple status-code checks to verify full response correctness. The framework's reusability was further validated by pointing the identical client architecture at a second, independent API (JSONPlaceholder) through a configuration change alone, with all 26 steps across 5 scenarios and 3 feature files passing (100% pass rate).

5.2 Future Enhancements

- Integrate with a CI/CD pipeline (GitHub Actions) for automated test runs on every push
- Extend coverage to additional resources on automationexercise.com, such as products and cart APIs
- Add parallel test execution support to reduce overall run time
- Introduce a real OAuth/token-based authentication flow against an API that supports it

5.3 Repository

The complete source code for this project will be published to a GitHub repository prior to final submission. Link:

https://github.com/Simantini06/Wipro_Selenium_Automation/tree/master/2_Capstone_Project/api-automation-framework